AI를 위한 PYTHON

PYTHON 함수

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수(Function)란?

반복되는 기능 코드를 하나로 묶어 이름을 붙인 것을 **함수**라 함

해당 기능 필요 시 **함수 이름을 불러서 사용** 가능

함수 호출

함수 호출이 되지 않으면

함수 코드는 실행(동작)하지 않음

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수(Function) 선언

```
def 함수명(매개변수1, 매개변수2,...., 매개변수N):
    실행문
    실행문
    return 반환값
```

함 수 명 ➔ 코드의 기능을 알 수 있도록 작성

매개 변수 ➔ 기능 구현에 필요한 **재료**로 **parameter**라 함

반 환 값 ➔ 기능 구현 후 **결과**물, 존재하지 않을 수도 있음

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수(Function) 사용 ➔ 함수 호출

parameter1, parameter2

함수명(매개변수1, 매개변수2)

결과 저장 변수 = 함수명(전달인자1, 전달인자2)

argument1, argument2

```
# 함수 선언
def add(a, b):
    value=a+b
    return value

# 함수 호출(사용)
result=add(10, 20)
print('add(10, 20) =', result)
```

a = 10
b = 20

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ return 반환

- 함수 종료 및 호출한 곳으로 돌아감
- 어떤 종류의 객체도 반환 가능
- return 사용하지 않거나 return만 적어도 함수 종료
- 함수 중간에 빠져나가기
- 여러 개 값 반환 가능

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 전달인자와 반환값 존재하는 경우

```
def sum(x, y):
    return x+y;

print(30,20)

print("Hello", "Tom")
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 전달인자만 존재하는 경우

```
def print_address(name):
    print("서울 특별시 종로구 1번지")
    print("파이썬 빌딩 7층")
    print(name)

print_address("홍길동")
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 전달인자, 반환값 없는 경우

```
def print_address( ):
    print("서울 특별시 종로구 1번지")
    print("파이썬 빌딩 7층")
    print("홍길동")

print_address()    # 함수 호출
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 전달인자, 반환값 모두 존재 하는 경우

```
def calculate_area (radius):
    area = 3.14 * radius**2
    return area

print( calculate_area (5.0) )
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 전달인자 개수 미정인 경우

```
def 함수명( *매개변수 ):
    실행문
    실행문
    return 반환값
```

가변인자

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 전달인자 개수 미정인 경우

```
def getSum(*args):
    sum=0
    for i in args:
        sum+=i
    return sum

result=getSum(1,2,3,4,5)
print("result %d" %result)

result=getSum(11,22,33,44,55,66,77,88,99)
print("result %d" %result)
```

가변인자
마지막에 위치

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 반환값이 여러 개 인 경우

```
def get_sum_mul(a, b):
    return a+b, a*b

result=get_sum_mul(2, 10)
print("get_sum_mul() result : " , result)

sum, mul =get_sum_mul(2, 10)
print("get_sum_mul() result - sum %d,  mul %d " %(sum, mul))
```

튜플 타입 리턴
언팩킹

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 키워드 파라미터 형태

```
def  함수명( **매개변수 ):
     실행문
     실행문
     return  반환값
```

key=value 형태 매개변수

딕셔너리 형태 반환

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 키워드 파라미터 형태

```
def print_kwargs( **kwargs ):
    print(kwargs)

# 함수 호출
print_kwargs(name='kim', age=30, gender='F')
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 매개변수 초기값 설정

```
def 함수명( 매개변수1, 매개변수2, 매재변수3=초기값):
    실행문
    실행문
    return 반환값
```

맨마직 위치!!

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 형태

- 매개변수 초기값 설정

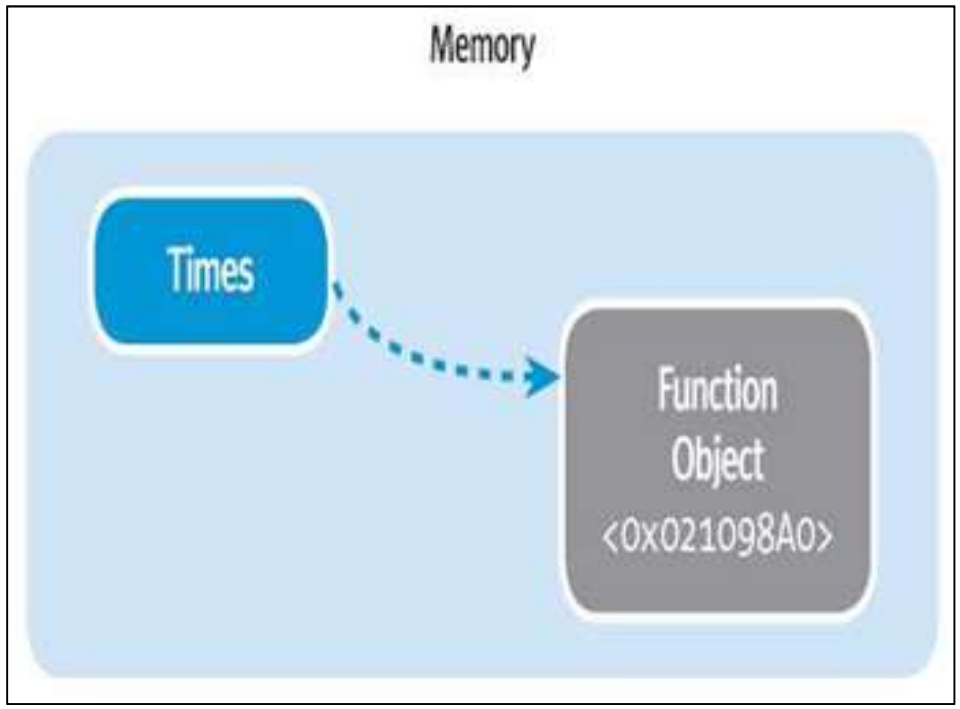
```
def set_info(name, old, man=True):
    print("Name is %s" %name)
    print("Old is %d" %old)

    if man:
        print("Man")
    else:
        print("Woman")

    return

set_info('Kim', 10)
set_info('Tom Lee', 15, False)
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 변수에 함수 담기

- 메모리에 객체 생성
- 레퍼런스 생성



Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 변수에 함수 담기

- 함수도 class ‘function’ 타입으로 변수에 대입 가능

```
def something(a):
    print(a)

p=something
print("₩ntype(p)=", type(p))

p(123)
p("Good Happy")
```

```
type(p)= <class 'function'>
123
Good Happy
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 변수에 함수 담기

```
def plus(a,b):
    return a*b

def minus(a,b):
    return a-b

first=[plus, minus]
print("₩ntype(first)=", type(first))

print("first[0](1,2)=", first[0](1,2))
print("first[1](1,2)=", first[1](1,2))
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수와 변수 관계

- 변수 위치에 따른 분류

지역변수 (Local Variable)

- 함수 안에 존재하는 함수
- 함수에서만 사용 가능
- 함수 실행 후 메모리에서 사라짐

전역변수 (Global Variable)

- 함수 밖에 존재하는 함수
- 프로그램 어디서나 사용 가능
- 프로그램 종료 시 메모리에서 사라짐

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수와 변수 관계

- 동일이름 변수명

test.py

```
a=10                                # 전역변수

def test(a):
    a += 1                          # 지역변수
    print('a =>', a)


# 함수 호출(사용)
test(20)
print("a is %d" %a)
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수와 변수 관계

- 전역변수 사용 설정

test.py

```
a=10                          # 전역변수

def test( ):
    # global a                # 전역변수  사용 알림
     a += 1                   # 전역변수

# 함수 호출(사용)
test(a)
print("a is %d" %a)
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ 함수 안에 함수

```
# 함수 선언 ------------------------------------
def print_hello():
    hello = 'Hello, world!'
    def print_message():
        print(hello)
    print_message()

# 함수 호출 ------------------------------------
print_hello()
```

Copyright 2021. SoHyunKim. All rights reserved

PYTHON 함수

◆ Lambda(람다)

➢ 이름 없는 한 줄 함수 즉, **익명함수**

변수명 = lambda 인수1, 인수2, … : 실행코드 (리턴 값)

```
add=lambda x, y : x+y
print( add(1,2) )

cal=[ lambda x,y : x+y,  lambda x,y : x-y, lambda x,y:  x/y ]
print( cal[0](1,2) )
print( cal[1](1,2) )
print( cal[2](1,2) )
```

Copyright 2021. SoHyunKim. All rights reserved